Day 2

Agent 제어 흐름 설계 & 상태 관리

AI Agent 전문 개발 과정

2026 · AI 개발자 / 데이터 엔지니어 / 기술 리더 대상

오늘의 커리큘럼

Session 1 · 2h

Agent 4요소 구조 설계

Goal · Memory · Tool · Control Logic

Session 2 · 2h

LangGraph 제어 흐름 설계

Node · Edge · State · 조건 분기

Session 3 · 2h

Tool 호출 통제 & Validation

검증 · Fallback · 루프 방지

Session 4 · 2h

구조 리팩토링 & 확장성

분리 · 결합도 · Trace 설계

Session 1

Agent 4요소 구조 설계

Agent를 "제대로" 만든다는 것

~~"LLM + 몇 가지 Tool을 연결한다"~~

↓

Goal · Memory · Tool · Control Logic
4요소를 명확히 분리한다

Agent를 "제대로" 만든다는 것

~~"LLM + 몇 가지 Tool을 연결한다"~~

↓

Goal · Memory · Tool · Control Logic
4요소를 명확히 분리한다

⚠ 2026년 프로덕션 Agent 장애 원인 1위

제어 흐름 미설계 → 무한 루프 / 상태 추적 불가 / 버그 위치 불명

Agent 4요소

Agent 4요소

🎯 **Goal**

무엇을 달성해야 하는가

성공 조건 + 실패 조건 정의 필수

Agent 4요소

🎯 Goal

무엇을 달성해야 하는가

성공 조건 + 실패 조건 정의 필수

🧠 Memory

무엇을 기억하는가

Working · Episodic · Semantic · Procedural

Agent 4요소

🎯 Goal

무엇을 달성해야 하는가

성공 조건 + 실패 조건 정의 필수

🧠 Memory

무엇을 기억하는가

Working · Episodic · Semantic · Procedural

🔧 Tool

무엇을 할 수 있는가

원자적(atomic) 단일 역할

Agent 4요소

🎯 Goal

무엇을 달성해야 하는가

성공 조건 + 실패 조건 정의 필수

🧠 Memory

무엇을 기억하는가

Working · Episodic · Semantic · Procedural

🔧 Tool

무엇을 할 수 있는가

원자적(atomic) 단일 역할

⚙️ Control Logic

어떻게 판단하는가

Plan → Execute → Observe → Reflect

Goal: 잘못된 정의 vs 올바른 정의

나쁜 예

```
# 모호한 Goal
goal = "데이터를 분석해"
```

Agent가 성공 여부를 판단할 수 없음
→ 무한 루프 유발

좋은 예

```
goal = Goal(
    description="이상치 탐지",
    success_condition=(
        "이상치 목록 + 보고서 생성 완료"
    ),
    abort_condition=(
        "데이터 접근 실패 또는 3회 초과"
    ),
    max_steps=10
)
```

Goal: 잘못된 정의 vs 올바른 정의

나쁜 예

```
# 모호한 Goal
goal = "데이터를 분석해"
```

Agent가 성공 여부를 판단할 수 없음
→ 무한 루프 유발

좋은 예

```
goal = Goal(
    description="이상치 탐지",
    success_condition=(
        "이상치 목록 + 보고서 생성 완료"
    ),
    abort_condition=(
        "데이터 접근 실패 또는 3회 초과"
    ),
    max_steps=10
)
```

핵심 원칙: Goal은 반드시 **검증 가능(verifiable)**해야 한다

Memory 4 레이어

```
┌──────────────────────────────┐
│ Episodic Memory  (이번 세션)        │ ← 가장 빠름, 가장 휘발성
├──────────────────────────────┤
│ Working Memory   (현재 Task 상태)   │ ← Agent State에 보관 ★
├──────────────────────────────┤
│ Semantic Memory  (도메인 지식)      │ ← Vector DB, RAG
├──────────────────────────────┤
│ Procedural Memory (실행 패턴)       │ ← 프롬프트, Few-shot
└──────────────────────────────┘
```

Memory 4 레이어

```
┌──────────────────────────────┐
│ Episodic Memory  (이번 세션)  │ ← 가장 빠름, 가장 휘발성
├──────────────────────────────┤
│ Working Memory   (현재 Task 상태) │ ← Agent State에 보관 ★
├──────────────────────────────┤
│ Semantic Memory  (도메인 지식)  │ ← Vector DB, RAG
├──────────────────────────────┤
│ Procedural Memory (실행 패턴)  │ ← 프롬프트, Few-shot
└──────────────────────────────┘
```

Working Memory가 오늘의 핵심

Memory 4 레이어

```
┌──────────────────────────────┐
│ Episodic Memory  (이번 세션)  │ ← 가장 빠름, 가장 휘발성
├──────────────────────────────┤
│ Working Memory   (현재 Task 상태) │ ← Agent State에 보관 ★
├──────────────────────────────┤
│ Semantic Memory  (도메인 지식)  │ ← Vector DB, RAG
├──────────────────────────────┤
│ Procedural Memory (실행 패턴)  │ ← 프롬프트, Few-shot
└──────────────────────────────┘
```

Working Memory가 오늘의 핵심

→ Agent State에 **무엇을** 담을지 결정하는 것이 설계의 핵심

→ 너무 많으면 컨텍스트 오염 / 너무 적으면 판단 오류

Tool: 원자성(Atomicity)이 핵심

나쁜 설계

```python
def search_and_summarize(query):
    results = web_search(query)
    summary = llm_summarize(results)
    save_to_db(summary)  # 사이드 이펙트!
    return summary
```

하나의 Tool이 3가지 일을 함
→ 오류 위치 추적 불가

좋은 설계

```python
@tool
def web_search(query: str) -> list: ...

@tool
def summarize_text(text: str) -> str: ...

@tool
def save_result(key: str, value: str): ...
```

각 Tool은 하나의 역할
→ 독립 테스트 가능

Planner-Executor 패턴

Planner-Executor 패턴

Single-step vs Multi-step 판단

Single-step

Multi-step

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료

Multi-step

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요

Multi-step

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역

Multi-step

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

- → 외부 정보 수집 필요

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

- → 외부 정보 수집 필요
- → 이전 결과를 다음이 참조

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

- → 외부 정보 수집 필요
- → 이전 결과를 다음이 참조
- → 조사, 계획, 자동화

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

- → 외부 정보 수집 필요
- → 이전 결과를 다음이 참조
- → 조사, 계획, 자동화
- → 실패 복구 가능

Single-step vs Multi-step 판단

Single-step

- → LLM 1회 호출로 완료
- → 상태 관리 불필요
- → 분류, 요약, 번역
- → 비용 낮음

Multi-step

- → 외부 정보 수집 필요
- → 이전 결과를 다음이 참조
- → 조사, 계획, 자동화
- → 실패 복구 가능

판단 기준: "LLM 한 번 호출로 완료되는가?" → Yes이면 Single-step

퀴즈: Agent 4요소

"언제 멈추고 언제 재시도할지 결정하는" 요소는?

A. Goal

B. Memory

C. Tool

D. Control Logic

퀴즈: Agent 4요소

"언제 멈추고 언제 재시도할지 결정하는" 요소는?

A. Goal

B. Memory

C. Tool

D. Control Logic

✅

D. Control Logic

판단 흐름 전체를 관장 → 멈춤/재시도/중단 결정

Session 2

LangGraph 기반 제어 흐름 설계

LangGraph가 필요한 이유

LangGraph가 필요한 이유

순수 코드의 한계

분기가 3개를 넘으면 if-else 지옥 → 흐름 파악 불가

LangGraph가 필요한 이유

순수 코드의 한계

분기가 3개를 넘으면 if-else 지옥 → 흐름 파악 불가

재시도 로직

재시도 횟수를 추적하고 루프를 제어하려면 별도 State 관리 필요

LangGraph가 필요한 이유

순수 코드의 한계

분기가 3개를 넘으면 if-else 지옥 → 흐름 파악 불가

재시도 로직

재시도 횟수를 추적하고 루프를 제어하려면 별도 State 관리 필요

LangGraph의 해결책

그래프 구조 → 흐름이 명확 / State 내장 / 시각화 지원 / 체크포인트

Node · Edge · State

📦

Node

실행 단위

Python 함수

`State → dict`

→

Edge

흐름 연결

다음 Node 지정

고정 / 조건부

🗄️

State

공유 데이터

TypedDict

`Annotated` 병합

Node · Edge · State

📦

Node

실행 단위

Python 함수

`State → dict`

→

Edge

흐름 연결

다음 Node 지정

고정 / 조건부

🗄️

State

공유 데이터

TypedDict

`Annotated` 병합

모든 Node는 State를 받아서 State의 일부(dict)를 반환한다

State 설계

```python
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    # 단순 덮어쓰기 필드
    query: str
    status: str

    # 리스트 누적 필드
    # (병렬 Node가 동시에 업데이트해도 안전)
    messages: Annotated[list, operator.add]
    tool_calls: Annotated[list, operator.add]

    # 카운터 (누적 합산)
    retry_count: Annotated[int, operator.add]
```

State 설계

```python
from typing import TypedDict, Annotated
import operator

class AgentState(TypedDict):
    # 단순 덮어쓰기 필드
    query: str
    status: str

    # 리스트 누적 필드
    # (병렬 Node가 동시에 업데이트해도 안전)
    messages: Annotated[list, operator.add]
    tool_calls: Annotated[list, operator.add]

    # 카운터 (누적 합산)
    retry_count: Annotated[int, operator.add]
```

왜 Annotated? 병렬 Node가 동시에 같은 필드를 업데이트할 때 데이터 충돌 방지

Conditional Edge: 분기 설계

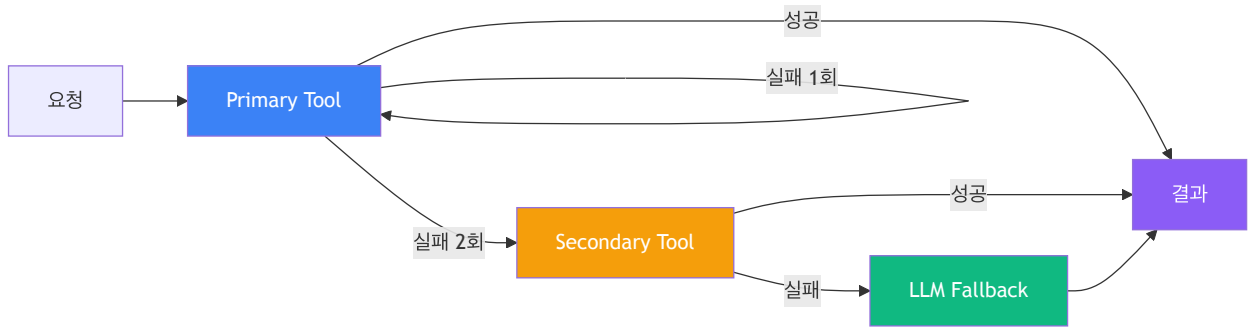
```python
def should_retry(state: WorkflowState) -> str:
    """반환값이 다음 Node 이름"""
    if state["retry_count"] ≥ 3:
        return "fail"
    if not state["search_results"]:
        return "retry"
    return "analyze"

graph.add_conditional_edges(
    "search",           # 이 Node 이후에 분기
    should_retry,       # 분기 함수
    {
        "analyze": "analyze",   # 반환값 → Node 이름
        "retry":   "search",    # 재시도
        "fail":    END,         # 종료
    }
)
```

Conditional Edge: 분기 설계

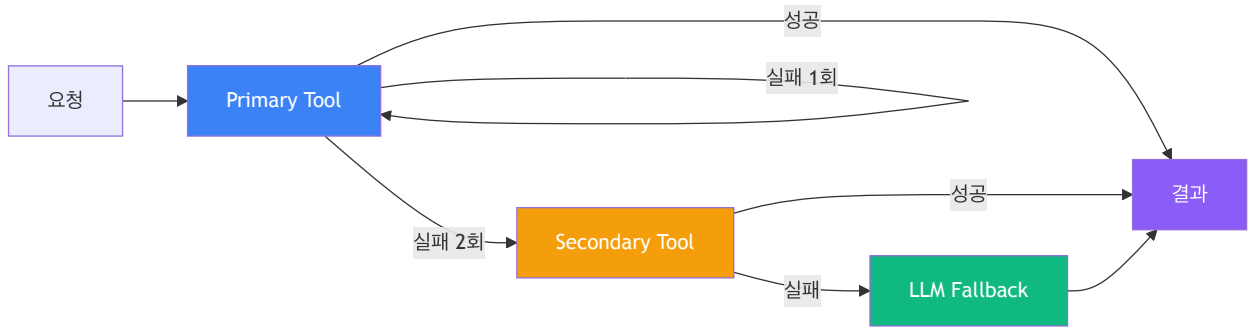
```python
def should_retry(state: WorkflowState) -> str:
    """반환값이 다음 Node 이름"""
    if state["retry_count"] ≥ 3:
        return "fail"
    if not state["search_results"]:
        return "retry"
    return "analyze"

graph.add_conditional_edges(
    "search",           # 이 Node 이후에 분기
    should_retry,       # 분기 함수
    {
        "analyze": "analyze",   # 반환값 → Node 이름
        "retry":   "search",    # 재시도
        "fail":    END,         # 종료
    }
)
```

주의: 분기 함수의 모든 가능한 반환값이 매핑에 포함되어야 함
누락 시 런타임 에러 발생

Retry / Fallback 전략

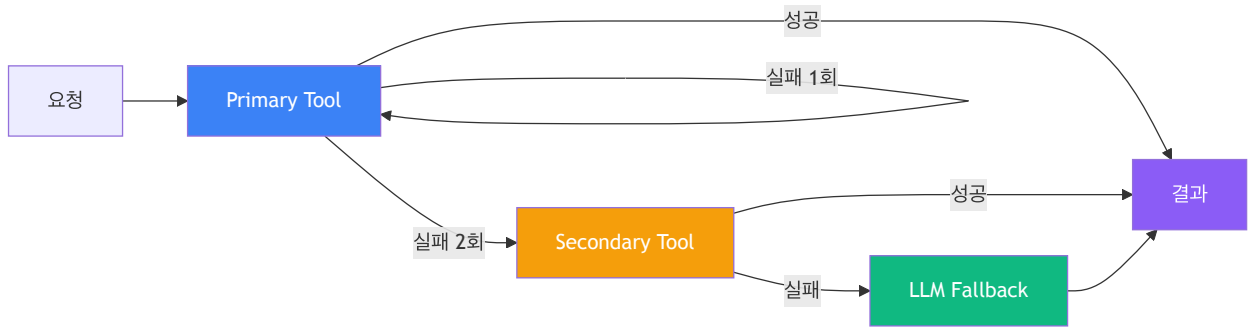


Retry / Fallback 전략



재시도: 동일 Tool, 동일 인자로 최대 N회 재시도

Retry / Fallback 전략



재시도: 동일 Tool, 동일 인자로 최대 N회 재시도

Fallback: 대체 Tool 또는 LLM 내부 지식으로 전환

State Propagation

```
Initial State:
{query: "AI 뉴스", results: [], analysis: "", retry: 0}
```

↓ search_node 실행 후

```
결과 병합:
{query: "AI 뉴스", results: ["뉴스1","뉴스2"], analysis: "", retry: 0}
```

↓ analyze_node 실행 후

```
결과 병합:
{query: "AI 뉴스", results: [...], analysis: "요약: ...", retry: 0}
```

State Propagation

```
Initial State:
{query: "AI 뉴스", results: [], analysis: "", retry: 0}
```

↓ search_node 실행 후

```
결과 병합:
{query: "AI 뉴스", results: ["뉴스1","뉴스2"], analysis: "", retry: 0}
```

↓ analyze_node 실행 후

```
결과 병합:
{query: "AI 뉴스", results: [...], analysis: "요약: ...", retry: 0}
```

Node가 반환하지 않은 필드는 이전 값이 그대로 유지됨

퀴즈: LangGraph

`Annotated[int, operator.add]` **로 선언된 retry_count가 현재 2일 때, Node가** `{"retry_count": 1}` **을 반환하면?**

A. 1 (덮어쓰기)

B. 2 (변화 없음)

C. 3 (합산)

D. 에러 발생

퀴즈: LangGraph

`Annotated[int, operator.add]` 로 선언된 retry_count가 현재 2일 때, Node가 `{"retry_count": 1}` 을 반환하면?

A. 1 (덮어쓰기)

B. 2 (변화 없음)

C. 3 (합산)

D. 에러 발생

✅

C. 3 (합산)

operator.add → 기존 2 + 반환값 1 = 3

Session 3

Tool 호출 통제 & Validation

Tool 호출: 가장 위험한 지점

Tool 호출: 가장 위험한 지점

외부 시스템 변경

이메일 발송, 파일 삭제, DB 변경 → 되돌릴 수 없음

Tool 호출: 가장 위험한 지점

외부 시스템 변경

이메일 발송, 파일 삭제, DB 변경 → 되돌릴 수 없음

비용 발생

API 호출, LLM 토큰 → 검증 없으면 무제한 소비

Tool 호출: 가장 위험한 지점

외부 시스템 변경

이메일 발송, 파일 삭제, DB 변경 → 되돌릴 수 없음

비용 발생

API 호출, LLM 토큰 → 검증 없으면 무제한 소비

잘못된 인자

LLM은 여전히 스키마에 맞지 않는 인자를 생성할 수 있음

Tool 호출: 가장 위험한 지점

외부 시스템 변경

이메일 발송, 파일 삭제, DB 변경 → 되돌릴 수 없음

비용 발생

API 호출, LLM 토큰 → 검증 없으면 무제한 소비

잘못된 인자

LLM은 여전히 스키마에 맞지 않는 인자를 생성할 수 있음

검증 없는 Tool 호출 = 프로덕션 장애의 직접 원인

Tool 스키마: LLM에게 명확히 알려라

```python
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(
        description=(
            "검색 키워드. 한국어 또는 영어. 최대 100자. "
            "LLM이 이미 알고 있는 일반 지식에는 사용 금지."
        ),
        max_length=100,
    )
    max_results: int = Field(
        default=5, ge=1, le=20,
        description="반환할 최대 결과 수 (1~20)",
    )

@tool(args_schema=SearchInput)
def web_search(query: str, max_results: int = 5) -> list[dict]:
    """웹에서 실시간 정보를 검색합니다.
    실시간 뉴스, 최신 정보, 특정 사실 확인에 사용하세요.
    """
    return search_api(query, max_results)
```

Tool 스키마: LLM에게 명확히 알려라

```python
from pydantic import BaseModel, Field
from langchain_core.tools import tool

class SearchInput(BaseModel):
    query: str = Field(
        description=(
            "검색 키워드. 한국어 또는 영어. 최대 100자. "
            "LLM이 이미 알고 있는 일반 지식에는 사용 금지."
        ),
        max_length=100,
    )
    max_results: int = Field(
        default=5, ge=1, le=20,
        description="반환할 최대 결과 수 (1~20)",
    )

@tool(args_schema=SearchInput)
def web_search(query: str, max_results: int = 5) -> list[dict]:
    """웹에서 실시간 정보를 검색합니다.
    실시간 뉴스, 최신 정보, 특정 사실 확인에 사용하세요.
    """
    return search_api(query, max_results)
```

명확한 description → LLM Tool 선택 정확도 20~40% 향상 (실험 결과)

검증 레이어 3단계

사전 검증 vs 사후 검증

사전 검증 (Pre-call)

```python
def pre_validate(tool, args, state):
    # 1. 권한 검증
    if not state.has_permission(tool):
        return deny("권한 없음")

    # 2. 의미 검증
    if tool == "send_email":
        if not is_valid_email(args["to"]):
            return deny("유효하지 않은 이메일")

    # 3. 중복 방지
    if is_duplicate(tool, args, state):
        return deny("동일 호출 반복")

    return allow()
```

사후 검증 (Post-call)

```python
def post_validate(tool, result, state):
    # 1. 타입 검증
    expected = TOOL_OUTPUT_TYPES[tool]
    if not isinstance(result, expected):
        return error("타입 불일치")

    # 2. 빈 결과 처리
    if not result:
        return empty(should_retry=True)

    # 3. 품질 검증
    if tool == "web_search":
        if len(result) < 2:
            return low_quality("결과 부족")

    return ok(result)
```

무한 루프 방지: 3가지 패턴

무한 루프 방지: 3가지 패턴

패턴 1: 동일 Tool 반복 호출

`search("AI") → 실패 → search("AI") → 실패 → ...`

무한 루프 방지: 3가지 패턴

패턴 1: 동일 Tool 반복 호출

`search("AI") → 실패 → search("AI") → 실패 → ...`

패턴 2: Tool 간 순환

`analyze() → needs_data → fetch() → needs_analysis → analyze() → ...`

무한 루프 방지: 3가지 패턴

패턴 1: 동일 Tool 반복 호출

`search("AI") → 실패 → search("AI") → 실패 → ...`

패턴 2: Tool 간 순환

`analyze() → needs_data → fetch() → needs_analysis → analyze() → ...`

패턴 3: 목표 달성 불가 루프

`write() → quality_check() → revision_needed → write() → ...`

무한 루프 방지: 3가지 패턴

패턴 1: 동일 Tool 반복 호출

`search("AI") → 실패 → search("AI") → 실패 → ...`

패턴 2: Tool 간 순환

`analyze() → needs_data → fetch() → needs_analysis → analyze() → ...`

패턴 3: 목표 달성 불가 루프

`write() → quality_check() → revision_needed → write() → ...`

방지책: max_steps + 동일 호출 횟수 추적 + 동일 인자 반복 탐지

Fallback 체인 구현

```python
TOOL_FALLBACK_CHAIN = {
    "search_api_premium": [
        "search_api_free",
        "llm_knowledge",
    ],
}

def execute_with_fallback(
    tool_name: str,
    args: dict,
) → Any:
    chain = [tool_name] + TOOL_FALLBACK_CHAIN.get(
        tool_name, []
    )

    for tool in chain:
        try:
            return TOOLS[tool](**args)
        except ToolError as e:
            log.warning(f"{tool} 실패: {e}")
            continue

    # 모든 Fallback 소진
    raise AllToolsFailedError(f"{tool_name} 및 Fallback 모두 실패")
```

Fallback 체인 구현

```python
TOOL_FALLBACK_CHAIN = {
    "search_api_premium": [
        "search_api_free",
        "llm_knowledge",
    ],
}

def execute_with_fallback(
    tool_name: str,
    args: dict,
) → Any:
    chain = [tool_name] + TOOL_FALLBACK_CHAIN.get(
        tool_name, []
    )

    for tool in chain:
        try:
            return TOOLS[tool](**args)
        except ToolError as e:
            log.warning(f"{tool} 실패: {e}")
            continue

    # 모든 Fallback 소진
    raise AllToolsFailedError(f"{tool_name} 및 Fallback 모두 실패")
```

퀴즈: Tool 검증

다음 중 "의미 검증(semantic validation)"에 해당하는 것은?

A. max_results가 정수(int)인지 확인

B. email 필드가 '@'를 포함하는지 확인

C. query 필드의 길이가 100자 이하인지 확인

D. 필수 필드가 모두 존재하는지 확인

퀴즈: Tool 검증

다음 중 "의미 검증(semantic validation)"에 해당하는 것은?

A. max_results가 정수(int)인지 확인

B. email 필드가 '@'를 포함하는지 확인

C. query 필드의 길이가 100자 이하인지 확인

D. 필수 필드가 모두 존재하는지 확인

✅

B. 이메일 형식 확인

A, C, D는 형식(스키마) 검증 / B는 의미(비즈니스 규칙) 검증

Session 4

구조 리팩토링 & 확장성 설계

단일 Agent의 한계

단일 함수 구조

```
def run_agent(input, db, llm, cache, slack):
    plan = llm.invoke(f"Plan: {input}")
    results = []
    for step in parse(plan):
        if "search" in step:
            r = requests.get( ... )  # 직접 연결
            db.save(r)               # 숨겨진 사이드 이펙트
            results.append(r)
        elif "analyze" in step:
            r = llm.invoke(results)
            results.append(r)
    return llm.invoke(f"Summary: {results}")
```

단일 Agent의 한계

단일 함수 구조

```
def run_agent(input, db, llm, cache, slack):
    plan = llm.invoke(f"Plan: {input}")
    results = []
    for step in parse(plan):
        if "search" in step:
            r = requests.get( ... )  # 직접 연결
            db.save(r)               # 숨겨진 사이드 이펙트
            results.append(r)
        elif "analyze" in step:
            r = llm.invoke(results)
            results.append(r)
    return llm.invoke(f"Summary: {results}")
```

God Function: 모든 것이 한 곳에

단일 Agent의 한계

단일 함수 구조

```
def run_agent(input, db, llm, cache, slack):
    plan = llm.invoke(f"Plan: {input}")
    results = []
    for step in parse(plan):
        if "search" in step:
            r = requests.get( ... )  # 직접 연결
            db.save(r)               # 숨겨진 사이드 이펙트
            results.append(r)
        elif "analyze" in step:
            r = llm.invoke(results)
            results.append(r)
    return llm.invoke(f"Summary: {results}")
```

God Function: 모든 것이 한 곳에

숨겨진 사이드 이펙트: db.save() 숨겨짐

단일 Agent의 한계

단일 함수 구조

```
def run_agent(input, db, llm, cache, slack):
    plan = llm.invoke(f"Plan: {input}")
    results = []
    for step in parse(plan):
        if "search" in step:
            r = requests.get( ... )  # 직접 연결
            db.save(r)               # 숨겨진 사이드 이펙트
            results.append(r)
        elif "analyze" in step:
            r = llm.invoke(results)
            results.append(r)
    return llm.invoke(f"Summary: {results}")
```

God Function: 모든 것이 한 곳에

숨겨진 사이드 이펙트: db.save() 숨겨짐

직접 결합: requests를 내부에서 직접 호출

단일 Agent의 한계

단일 함수 구조

```
def run_agent(input, db, llm, cache, slack):
    plan = llm.invoke(f"Plan: {input}")
    results = []
    for step in parse(plan):
        if "search" in step:
            r = requests.get( ... )  # 직접 연결
            db.save(r)               # 숨겨진 사이드 이펙트
            results.append(r)
        elif "analyze" in step:
            r = llm.invoke(results)
            results.append(r)
    return llm.invoke(f"Summary: {results}")
```

God Function: 모든 것이 한 곳에

숨겨진 사이드 이펙트: db.save() 숨겨짐

직접 결합: requests를 내부에서 직접 호출

상태 추적 불가: 중간 결과 알 수 없음

전환 기준 & 절차

전환 시점

전환 절차

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때

전환 절차

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때

전환 절차

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때

전환 절차

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

1. → Step 1: 책임 분리 (Planner/Executor)

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

1. → Step 1: 책임 분리 (Planner/Executor)
2. → Step 2: State 명시적 정의

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

1. → Step 1: 책임 분리 (Planner/Executor)
2. → Step 2: State 명시적 정의
3. → Step 3: Tool을 독립 모듈로 분리

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

1. → Step 1: 책임 분리 (Planner/Executor)
2. → Step 2: State 명시적 정의
3. → Step 3: Tool을 독립 모듈로 분리
4. → Step 4: LangGraph 구조로 전환

전환 기준 & 절차

전환 시점

- → Tool이 3개를 초과할 때
- → 분기 조건이 2개를 초과할 때
- → 실행 시간이 10초를 초과할 때
- → 팀원이 이해하기 어렵다고 할 때

전환 절차

1. → Step 1: 책임 분리 (Planner/Executor)
2. → Step 2: State 명시적 정의
3. → Step 3: Tool을 독립 모듈로 분리
4. → Step 4: LangGraph 구조로 전환

원칙: 조기 최적화를 피한다. 단순하게 시작하고 필요할 때 확장한다.

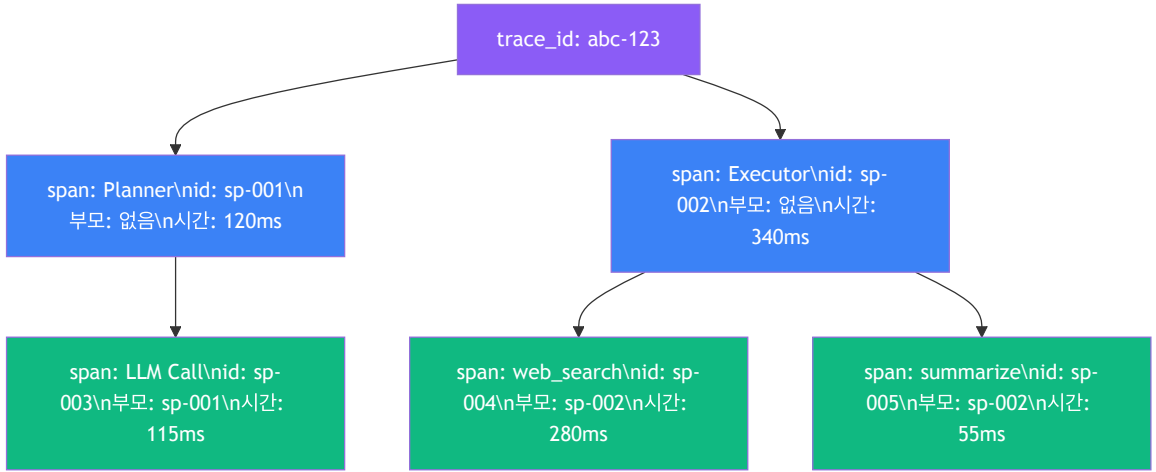
결합도 낮추기: 인터페이스 추상화

```python
from abc import ABC, abstractmethod

# 인터페이스: 구체 구현이 아닌 계약
class LLMProvider(ABC):
    @abstractmethod
    def complete(self, prompt: str) -> str: ...

# 구현 A
class OpenAIProvider(LLMProvider):
    def complete(self, prompt: str) -> str:
        return openai_client.chat(prompt)

# 구현 B
class AnthropicProvider(LLMProvider):
    def complete(self, prompt: str) -> str:
        return anthropic_client.messages(prompt)

# 사용자: 인터페이스에만 의존
class Planner:
    def __init__(self, llm: LLMProvider):  # 구체 구현 모름
        self.llm = llm

    def plan(self, query: str) -> list[Step]:
        return self.llm.complete(f"Plan: {query}")
```

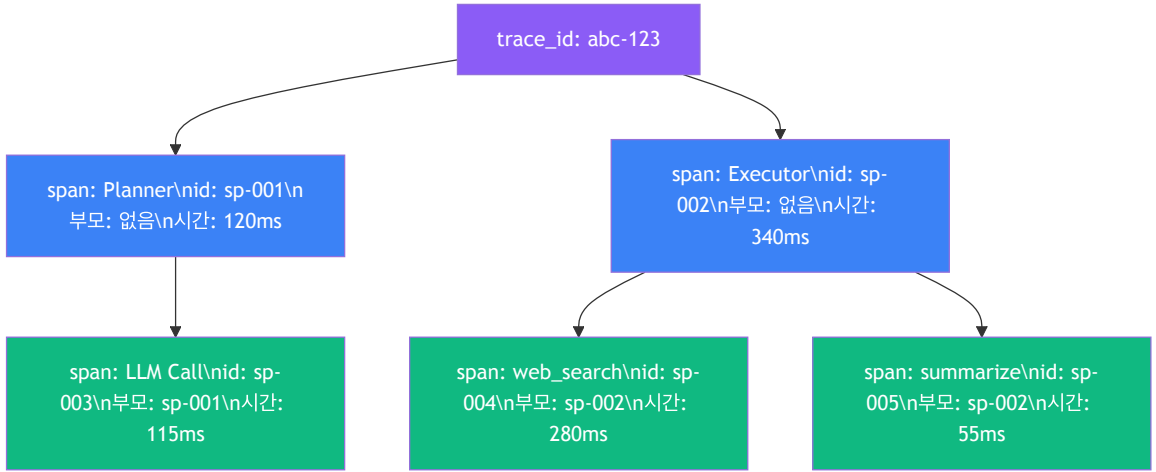
결합도 낮추기: 인터페이스 추상화

```python
from abc import ABC, abstractmethod

# 인터페이스: 구체 구현이 아닌 계약
class LLMProvider(ABC):
    @abstractmethod
    def complete(self, prompt: str) -> str: ...

# 구현 A
class OpenAIProvider(LLMProvider):
    def complete(self, prompt: str) -> str:
        return openai_client.chat(prompt)

# 구현 B
class AnthropicProvider(LLMProvider):
    def complete(self, prompt: str) -> str:
        return anthropic_client.messages(prompt)

# 사용자: 인터페이스에만 의존
class Planner:
    def __init__(self, llm: LLMProvider):  # 구체 구현 모름
        self.llm = llm

    def plan(self, query: str) -> list[Step]:
        return self.llm.complete(f"Plan: {query}")
```

Trace 로그: 관찰 가능성의 기본



Trace 로그: 관찰 가능성의 기본



span_id + parent_id → 인과관계 추적 가능 → 프로덕션 디버깅 가능

유지보수 가능한 구조: 체크리스트

유지보수 가능한 구조: 체크리스트

✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다

유지보수 가능한 구조: 체크리스트

- ✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다
- ✓ **테스트 가능**: 외부 의존 없이 단위 테스트가 가능하다

유지보수 가능한 구조: 체크리스트

- ✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다
- ✓ **테스트 가능**: 외부 의존 없이 단위 테스트가 가능하다
- ✓ **명확한 인터페이스**: 함수 시그니처만 보면 역할을 안다

유지보수 가능한 구조: 체크리스트

- ✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다
- ✓ **테스트 가능**: 외부 의존 없이 단위 테스트가 가능하다
- ✓ **명확한 인터페이스**: 함수 시그니처만 보면 역할을 안다
- ✓ **상태 추적**: 임의 시점의 State를 재현할 수 있다

유지보수 가능한 구조: 체크리스트

- ✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다
- ✓ **테스트 가능**: 외부 의존 없이 단위 테스트가 가능하다
- ✓ **명확한 인터페이스**: 함수 시그니처만 보면 역할을 안다
- ✓ **상태 추적**: 임의 시점의 State를 재현할 수 있다
- ✓ **교체 가능**: LLM Provider를 바꿔도 다른 코드 수정이 없다

유지보수 가능한 구조: 체크리스트

- ✓ **단일 책임**: 각 클래스가 한 가지 이유로만 변경된다
- ✓ **테스트 가능**: 외부 의존 없이 단위 테스트가 가능하다
- ✓ **명확한 인터페이스**: 함수 시그니처만 보면 역할을 안다
- ✓ **상태 추적**: 임의 시점의 State를 재현할 수 있다
- ✓ **교체 가능**: LLM Provider를 바꿔도 다른 코드 수정이 없다
- ✓ **관찰 가능**: 프로덕션에서 무슨 일이 일어나는지 로그로 안다

코드 스멜 vs 개선 패턴

코드 스멜

```
# 스멜 1: God Function
def process_all(input, db, llm, cache, slack, email):
    ...  # 100줄

# 스멜 2: 전역 상태
_global_ctx = {}
def node_a():
    _global_ctx["result"] = llm.invoke( ... )

# 스멜 3: 숨겨진 사이드 이펙트
def analyze(text: str) → str:
    db.save(text)   # 호출자가 모름
    return llm(text)
```

개선 패턴

```
# 개선 1: 단일 책임 클래스
class Planner:  ...
class Executor:  ...

# 개선 2: 명시적 State
class AgentState(TypedDict):
    result: str

# 개선 3: 명시적 의존성
def analyze(
    text: str,
    *,
    persist: bool = False,  # 명시적
) → AnalysisResult:
    result = AnalysisResult(llm(text))
    return result
```

Day 2 핵심 요약

Day 2 핵심 요약

Agent 4요소

Goal(검증 가능) + Memory(Working★) + Tool(원자적) + Control(PEAR 사이클)

Day 2 핵심 요약

Agent 4요소

Goal(검증 가능) + Memory(Working★) + Tool(원자적) + Control(PEAR 사이클)

LangGraph

Node(State→dict) + Edge(조건 분기) + State(Annotated 병합) + 체크포인트

Day 2 핵심 요약

Agent 4요소

Goal(검증 가능) + Memory(Working★) + Tool(원자적) + Control(PEAR 사이클)

LangGraph

Node(State→dict) + Edge(조건 분기) + State(Annotated 병합) + 체크포인트

Tool 검증

Schema 검증 → 의미 검증 → 실행 → 결과 검증 + 루프 방지 + Fallback

Day 2 핵심 요약

Agent 4요소

Goal(검증 가능) + Memory(Working★) + Tool(원자적) + Control(PEAR 사이클)

LangGraph

Node(State→dict) + Edge(조건 분기) + State(Annotated 병합) + 체크포인트

Tool 검증

Schema 검증 → 의미 검증 → 실행 → 결과 검증 + 루프 방지 + Fallback

확장성 설계

인터페이스 추상화 + State 중계 + Trace 로그 + 단계적 전환

종합 퀴즈: 설계 판단

다음 상황에서 가장 적절한 조치는?

프로덕션 Agent가 web_search Tool을 5회 연속 동일 인자로 호출했다. 각 호출은 성공(HTTP 200)했지만 결과가 비어 있다.

A. max_results 파라미터를 늘린다

B. 루프 방지 로직이 없으므로 추가한다 + Fallback Tool을 연결한다

C. LLM 모델을 더 좋은 것으로 교체한다

D. 재시도 횟수를 10회로 늘린다

종합 퀴즈: 설계 판단

다음 상황에서 가장 적절한 조치는?

프로덕션 Agent가 web_search Tool을 5회 연속 동일 인자로 호출했다. 각 [illegible] (HTTP 200)했지만 결과가 비어 있다.

A. max_results 파라미터를 늘린다

B. 루프 방지 로직이 없으므로 추가한다 + Fallback Tool을 연결한다

C. LLM 모델을 더 좋은 것으로 교체한다

D. 재시도 횟수를 10회로 늘린다

✅

B

동일 인자 반복 = 루프 패턴

루프 방지 + Fallback이 올바른 해결책

A,D는 루프를 악화시킴 / C는 근본 원인 아님

Day 2 실습 로드맵

1. **Session 1 실습: Agent 상태 다이어그램 작성**
   코드 리뷰 Agent의 4요소를 설계하고 Python으로 구현 (90분)

2. **Session 2 실습: LangGraph 워크플로우 구현**
   데이터 파이프라인 Agent를 Node-Edge-State로 구현 (90분)

3. **Session 3 실습: Tool Fallback 구현**
   멀티 소스 정보 수집 Agent의 검증 시스템 구현 (90분)

4. **Session 4 실습: 구조 리팩토링**
   단일 Agent를 확장형 구조로 리팩토링 + Trace 추가 (90분)

실습 환경 준비 중...

환경 설정

```
# 필요 패키지 설치
pip install langgraph langchain-openai \
            langchain-community pydantic

# 환경 변수 설정
export OPENAI_API_KEY="your-key-here"

# 실습 파일 확인
ls labs/
# day2-langgraph-workflow/
# day2-tool-validation/
```

환경 설정

```bash
# 필요 패키지 설치
pip install langgraph langchain-openai \
            langchain-community pydantic

# 환경 변수 설정
export OPENAI_API_KEY="your-key-here"

# 실습 파일 확인
ls labs/
# day2-langgraph-workflow/
# day2-tool-validation/
```

```python
# 설치 확인
import langgraph
import langchain_openai
print("LangGraph:", langgraph.__version__)
# LangGraph: 0.3.x
```

환경 설정

```
# 필요 패키지 설치
pip install langgraph langchain-openai \
            langchain-community pydantic

# 환경 변수 설정
export OPENAI_API_KEY="your-key-here"

# 실습 파일 확인
ls labs/
# day2-langgraph-workflow/
# day2-tool-validation/
```

```
# 설치 확인
import langgraph
import langchain_openai
print("LangGraph:", langgraph.__version__)
# LangGraph: 0.3.x
```

설치 완료 후 실습 디렉토리로 이동하여 시작합니다

실습 1: LangGraph Workflow

목표

Node–Edge–State 구조를 이해하고 조건 분기와 재시도 로직이 포함된 워크플로우를 구현한다

I DO (15분)

뉴스 요약 워크플로우 라이브 코딩

WE DO (30분)

코드 리뷰 워크플로우 함께 구현

YOU DO (45분)

데이터 파이프라인 Agent 독립 구현

실습 1: LangGraph Workflow

목표

Node–Edge–State 구조를 이해하고 조건 분기와 재시도 로직이 포함된 워크플로우를 구현한다

I DO (15분)
뉴스 요약 워크플로우 라이브 코딩

WE DO (30분)
코드 리뷰 워크플로우 함께 구현

YOU DO (45분)
데이터 파이프라인 Agent 독립 구현

```python
# 시작점: labs/day2-langgraph-workflow/src/
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator

class PipelineState(TypedDict):
    csv_url: str
    data: list
    stats: dict
    # TODO: 나머지 필드 추가
```

실습 2: Tool Validation

목표

Tool 호출 전후 검증과 Fallback 체인을 구현하여 안정적인 Tool 실행 시스템을 만든다

I DO (15분)

안전한 웹 검색 Tool 라이브 구현

WE DO (30분)

데이터 처리 Tool 시스템 함께 구현

YOU DO (45분)

멀티 소스 정보 수집 Agent 독립 구현

실습 2: Tool Validation

목표

Tool 호출 전후 검증과 Fallback 체인을 구현하여 안정적인 Tool 실행 시스템을 만든다

I DO (15분)
안전한 웹 검색 Tool 라이브 구현

WE DO (30분)
데이터 처리 Tool 시스템 함께 구현

YOU DO (45분)
멀티 소스 정보 수집 Agent 독립 구현

```python
# 시작점: labs/day2-tool-validation/src/
from pydantic import BaseModel, Field

class ToolOrchestrator:
    def __init__(self, tools, max_steps=20):
        self.tools = tools
        self.max_steps = max_steps

    def execute(self, tool_name, args, state):
        # TODO: 사전 검증 → 실행 → 사후 검증
        pass
```

Day 2 마무리

"구조가 Agent의 수명을 결정한다"

오늘 배운 것

- → 4요소로 Agent를 구조화하는 방법
- → LangGraph로 흐름을 명확히 표현하는 방법
- → Tool 호출을 안전하게 통제하는 방법
- → 확장 가능한 구조로 전환하는 방법

내일 Day 3 예고

- → Multi-Agent 시스템 설계
- → Agent 간 통신 프로토콜
- → 오케스트레이터 패턴
- → 분산 State 관리